VLSI LAB FINAL PROJECT

From NAND Gates to an 8-bit Accumulator CPU

A step-by-step dataflow, circuit hierarchy, reset/edge guide, implementation pipeline and viva preparation

Project detail	Your assignment
Student ID	22201054
Last digit / assigned set	4 / Set 4
Mandatory instructions	LDI, ADD
Assigned instructions	STA, LDR, AND
Complete instruction set	LDI, ADD, STA, LDR, AND
Course	CSE 450 — VLSI Lab

How to use this guide: Follow the hierarchy in order. First understand the small gates, then the storage elements and arithmetic blocks, then how the CPU connects them. The final sections give a presentation-ready pipeline and viva questions.

Important distinction: Your uploaded assignment requires a modular VHDL CPU (PC, ROM, control unit, ALU and ACC/R/Z). Building every logic gate physically from NAND gates is a useful conceptual learning path, but the brief does not say that you must implement a transistor-level NAND-only CPU. This guide covers both the conceptual gate hierarchy and the actual required CPU architecture.

1. Your assignment — what exactly must be built?

The supplied project brief says that every student implements the two mandatory instructions **LDI** and **ADD**, plus three instructions selected by the last digit of the student ID. Your ID ends in 4, so your set is Set 4.

Instruction	Opcode	Meaning	ACC / R / Z effect
LDI imm	0001	Load immediate constant	ACC ← imm; update Z
ADD imm	0010	Add immediate to accumulator	ACC ← ACC + imm (8-bit); update Z
STA	1010	Save accumulator in holding register	R ← ACC; ACC and Z unchanged
LDR	1011	Load saved value into accumulator	ACC ← R; update Z
AND imm	0100	Bitwise AND with immediate	ACC ← ACC AND imm; update Z

CPU widths: PC is 4 bits; ROM has 16 locations, each 12 bits; instruction opcode is 4 bits; operand is 8 bits; ACC and R are 8 bits; Z is 1 bit. The top-level outputs include Q[7:0] (ACC), Z and PC[3:0].

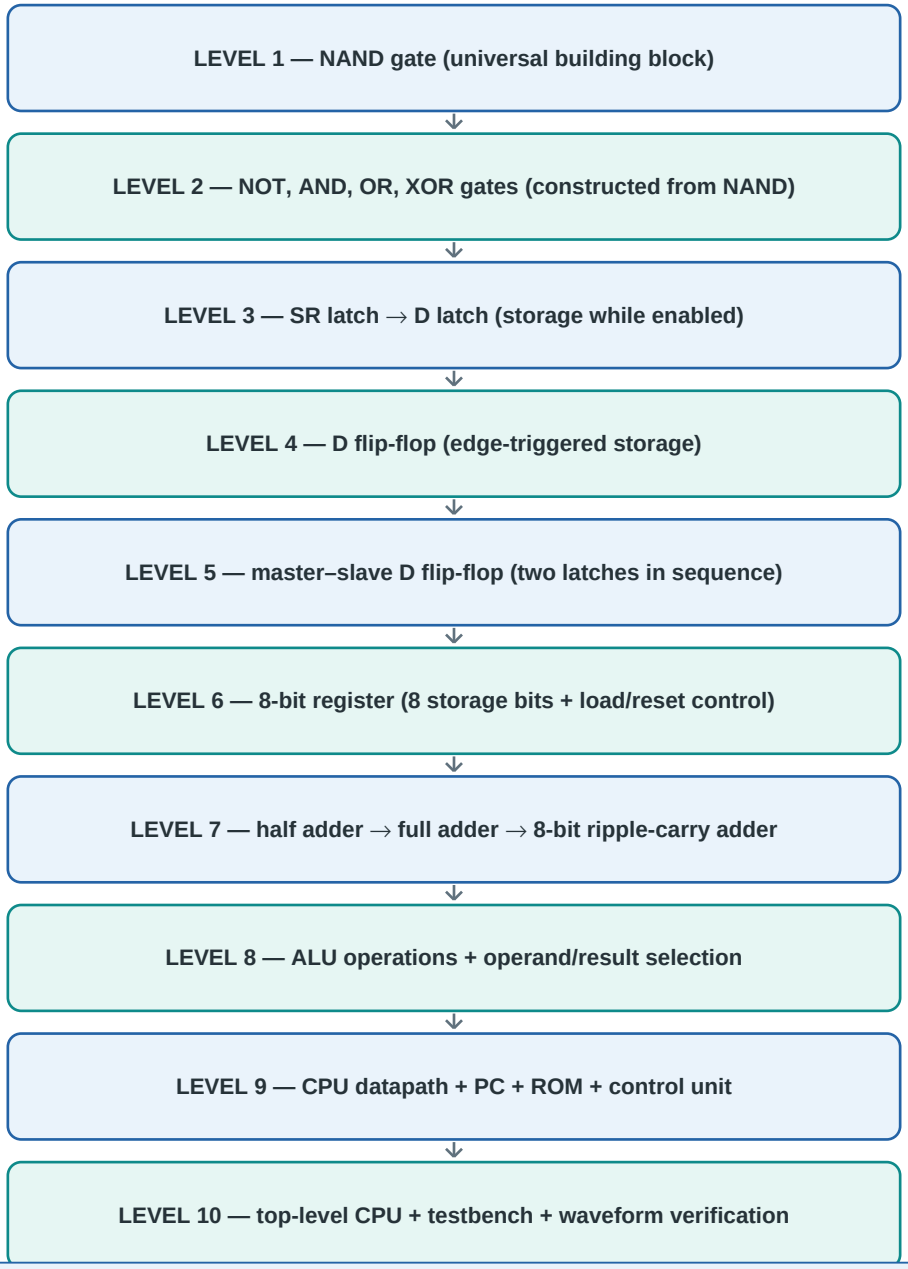
Instruction encoding: instr[11:8] = opcode and instr[7:0] = operand. For example, LDI 0x1E is opcode 0001 followed by operand 00011110, producing 0x11E.

Required modules from the brief



2. The complete learning hierarchy: from NAND to CPU

Think of the design as reusable layers. Each higher block uses lower blocks that have already been understood or built. A gate is not copied by pasting a name into a circuit; it is instantiated/connected as a component in a structural design, or described behaviorally in VHDL where the assignment permits it.



Reuse rule: If you already have a verified 1-bit block, scale it by instantiating multiple copies and connecting the signals correctly. Examples: 8 one-bit storage cells make an 8-bit register; 8 one-bit full adders make an 8-bit ripple-carry adder. Not every block scales in exactly the same way: a 1-bit full adder scales naturally to a multi-bit adder, while a latch or flip-flop needs shared control signals and carefully defined timing.

What should be reused where?

Higher-level block	Reuse these lower-level blocks
AND / OR / NOT / XOR	NAND gate(s)
D latch	NAND-based SR latch + input gating/inversion
D flip-flop	Two level-sensitive latches, opposite phases (conceptual master–slave)

Higher-level block	Reuse these lower-level blocks
8-bit register	8 D flip-flops + common CLK, RESET and load/enable logic
Half adder	XOR + AND
Full adder	2 half-adder stages + OR, or equivalent XOR/AND network
8-bit adder	8 full adders connected by carry chain (or an equivalent arithmetic operator in VHDL)
CPU ALU	Adder/subtractor if needed, bitwise gates, result mux/selection
Mini CPU	PC + ROM + control + ALU/datapath + ACC/R/Z registers

3. NAND gate and the basic logic gates

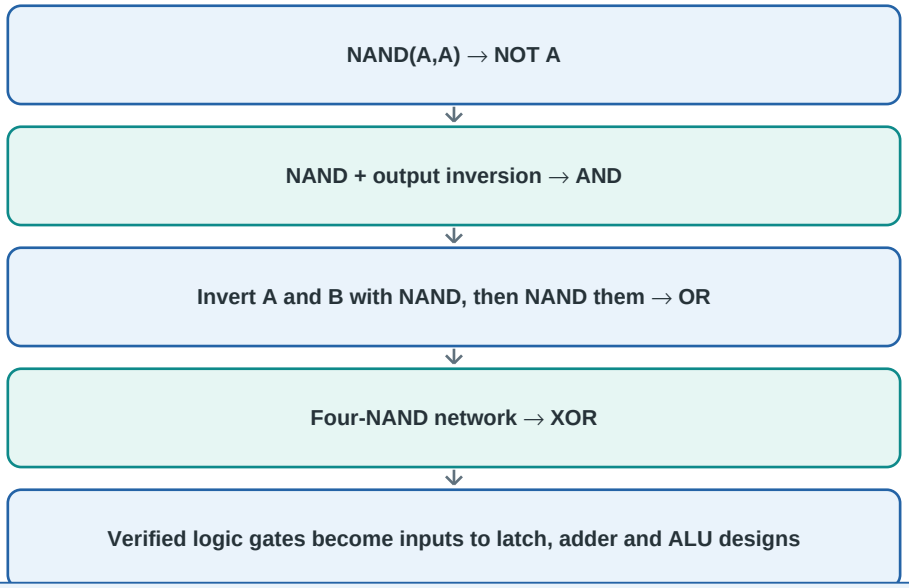
A NAND gate outputs 0 only when both inputs are 1. Its Boolean expression is Y = NOT(A AND B). It is called a universal gate because combinations of NAND gates can implement NOT, AND, OR and XOR, and therefore can be used to build much more complex combinational logic.

A	B	NAND	AND	OR	XOR
0	0	1	0	0	0
0	1	1	0	1	1
1	0	1	0	1	1
1	1	0	1	1	0

Construct each gate using NAND

Gate	Connection using NAND	Why it works
NOT A	N1 = NAND(A,A)	Tying both inputs together gives NOT A.
AND	N1 = NAND(A,B); Y = NAND(N1,N1)	Invert the NAND output.
OR	N1 = NAND(A,A); N2 = NAND(B,B); Y = NAND(N1,N2)	By De Morgan's law, NOT((NOT A) AND (NOT B)) = A OR B.
XOR	N1=NAND(A,B); N2=NAND(A,N1); N3=NAND(B,N1); Y=NAND(N2,N3)	Output is 1 when A and B differ.

Gate dependency diagram



Design check: In a truth table, verify every possible input combination. For 2-input gates there are only 4 rows. Do not trust a circuit just because it works for one test vector.

4. Storage begins: SR latch and D latch

Combinational gates calculate outputs from current inputs. Storage elements remember a value. This is the first important change in the pipeline: feedback is introduced so the circuit can preserve a state.

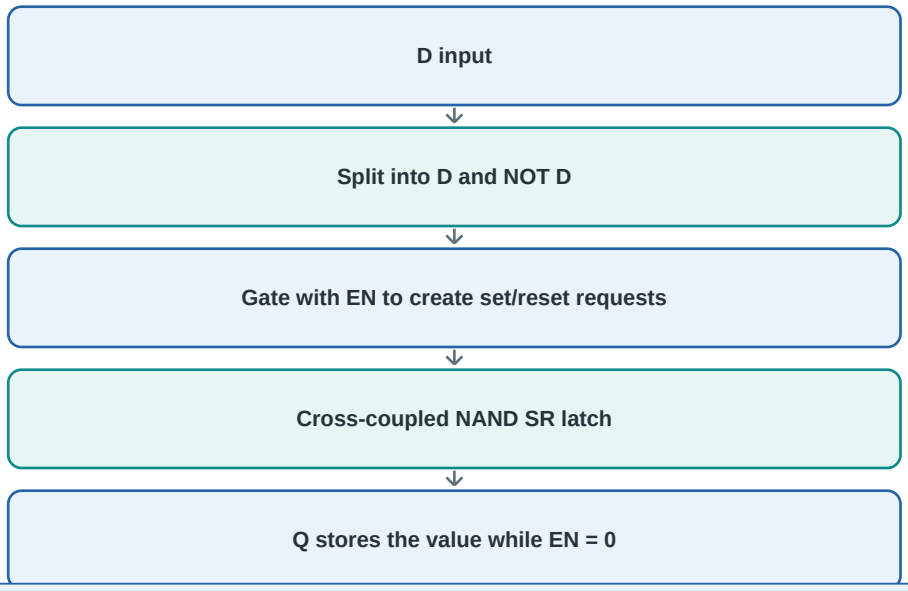
4.1 NAND SR latch

A basic NAND SR latch uses two cross-coupled NAND gates. Its inputs are typically active-low S■ and R■. Because they are active-low, S■=0 requests Set and R■=0 requests Reset. When both are 1, the latch holds its previous state. The condition S■=0 and R■=0 is invalid for the conventional NAND SR latch because it forces both outputs high and breaks the complementary-output expectation.

S■	R■	Action	Q(next)
1	1	Hold previous state	Q
0	1	Set	1
1	0	Reset	0
0	0	Forbidden / invalid	Do not use

4.2 D latch

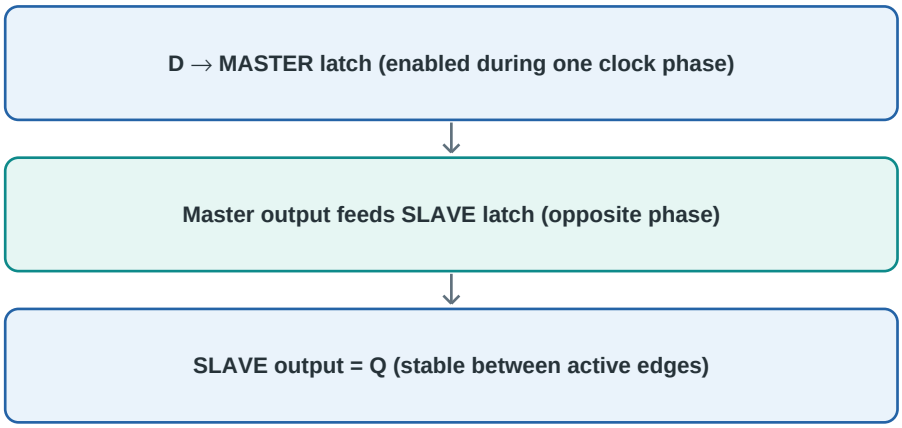
A D latch adds gating so one data input D controls the stored value. When Enable (EN)=1, the latch is transparent: Q follows D. When EN=0, it holds the last value. Internally, D and NOT D are arranged so the forbidden SR input combination is avoided in normal operation.



Key viva distinction: A latch is level-sensitive; a flip-flop is edge-triggered. A D latch can change while its enable is active, not only at a clock edge.

5. D flip-flop, master–slave design and edge timing

A D flip-flop stores D at a specified clock edge and holds Q between active edges. In a common conceptual master–slave implementation, two latches are connected in series and controlled by opposite clock phases. The master captures during one phase; the slave updates during the opposite phase. The combined behavior is edge-triggered.



Clock edge concepts

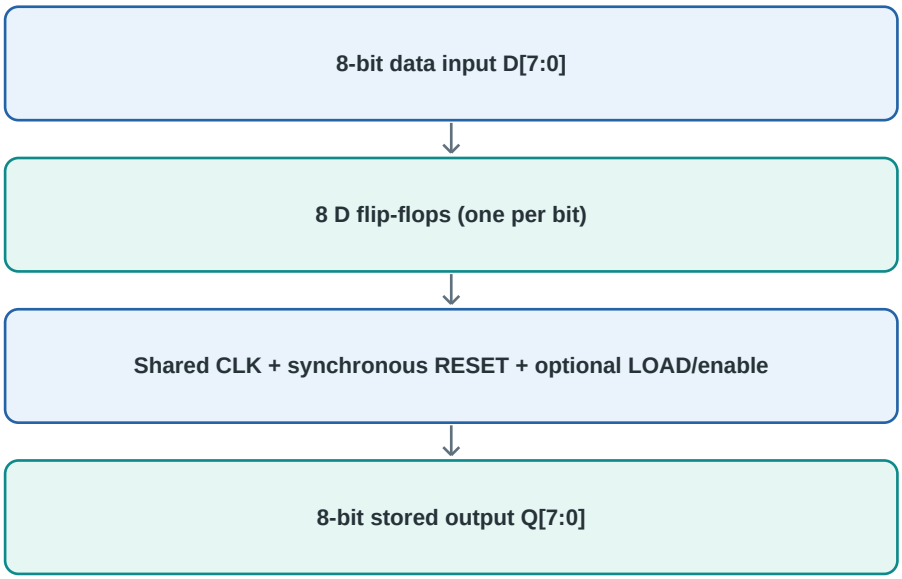
Term	Meaning	What to remember
Rising edge	CLK changes 0 → 1	This project updates state on this edge.
Falling edge	CLK changes 1 → 0	Not the active update edge in this assignment.
Setup time	D must be stable before the active edge	A real hardware timing requirement.
Hold time	D must remain stable briefly after the edge	A real hardware timing requirement.
Synchronous reset	Reset is checked at the active clock edge	RESET=1 between edges alone does not immediately clear state.

Project-specific rule: PC, ACC, R and Z update on the rising edge of CLK. RESET is synchronous and active-high. At a rising edge with RESET=1, the required state is PC=0, ACC=0x00, R=0x00, Z=1. RESET has priority over instruction execution.

Do not confuse reset behavior with initialization. A synchronous reset is a control event sampled on a clock edge; do not assume the register resets immediately when RESET rises between clock edges.

6. Registers: making one bit into eight bits

A 1-bit register stores one bit. An 8-bit register is eight 1-bit storage elements in parallel. All bits share the clock and reset. A load/enable signal determines whether the register accepts new data or keeps its old value.



Register behavior

RESET at rising edge	LOAD / enable	Next stored value
1	X	0x00 (reset priority)
0	1	D[7:0]
0	0	Hold current Q[7:0]

In the CPU, ACC is an 8-bit accumulator register and is also exposed as output Q[7:0]. R is an 8-bit holding register used by STA/LDR. Z is a single-bit flag. The PC is a 4-bit register.

Common mistake: Do not update ACC for STA. STA copies ACC into R and leaves ACC unchanged. Do not update Z for STA; the assignment explicitly says Z is unchanged for STA. LDR copies R into ACC and updates Z based on the loaded ACC value.

7. Half adder → full adder → 8-bit adder

Your reuse theory is correct for adders: first understand one bit, then chain multiple 1-bit stages. The important connection is the carry output of each lower bit to the carry input of the next higher bit.

7.1 Half adder (HA)

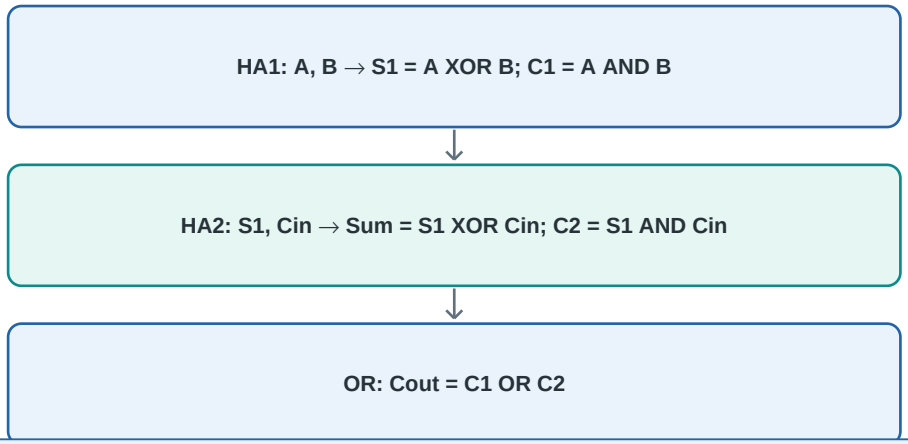
Inputs A and B; outputs Sum and Carry. It has no carry-in input.

Sum = A XOR B
Carry = A AND B

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

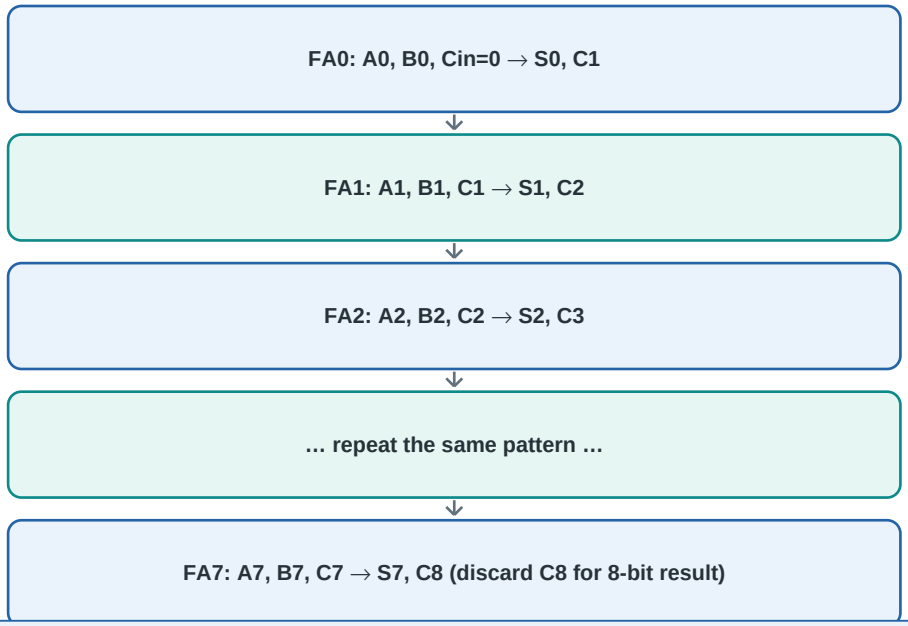
7.2 Full adder (FA)

Inputs A, B and carry-in Cin; outputs Sum and carry-out Cout. A full adder can be built from two half adders and one OR gate.



Equations: Sum = A XOR B XOR Cin
Cout = (A AND B) OR (Cin AND (A XOR B))

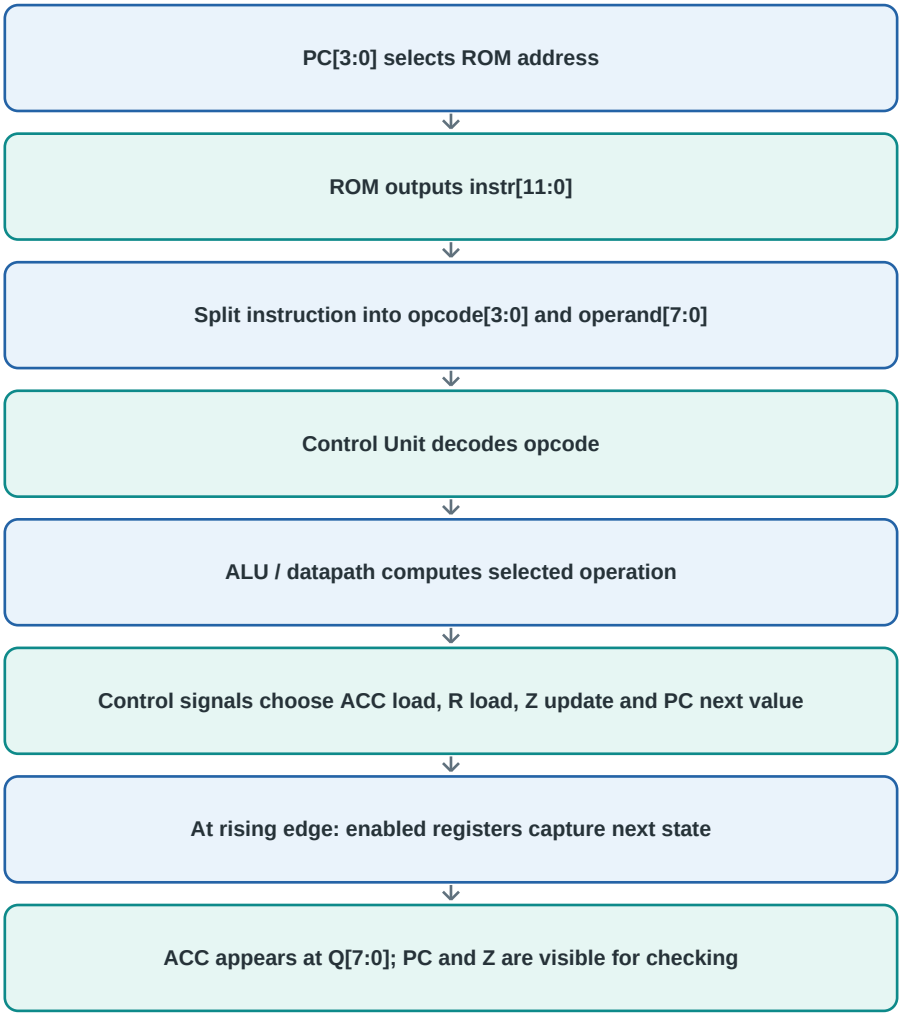
7.3 8-bit ripple-carry adder



For this CPU's ADD instruction, the result is 8-bit and carry is discarded. Example: 0xFF + 0x01 = 0x00 in ACC because the 9th carry bit is not stored. Z becomes 1 because the 8-bit result is zero.

8. CPU datapath: how the blocks work together

The CPU is a single-cycle accumulator CPU. Fetch, decode and execute are described as happening in the same instruction cycle; state changes are committed at the rising clock edge. ROM is combinationally read using PC, while PC, ACC, R and Z are sequential state.



Major modules and responsibilities

Module	Input(s)	Output / responsibility
PC	CLK, RESET, next-PC control	4-bit address; normally PC+1
Program ROM	PC address	12-bit instruction; constant 16-entry array
Control Unit	opcode, relevant flags	ALU select, register enables, PC control
ALU	ACC, operand, ALU select	Combinational result for supported operations
ACC register	result / R data / immediate path, enable, reset	ACC[7:0] / Q[7:0]
R register	ACC, STA enable, reset	Saved 8-bit value
Z flag	selected new ACC value, update enable, reset	1 when updated ACC value is 0x00
Top-level CPU	clock/reset and submodule wires	Connects all blocks and exposes outputs
Testbench	clock and reset stimuli	Checks PC, ACC, Z and R behavior

9. The exact control behavior for your five instructions

The control unit turns an opcode into enables and selects. The table below is a conceptual control checklist for Set 4. The exact signal names depend on your VHDL design, but the effects must match the assignment.

Instruction	ACC source / operation	ACC write?	R write?	Z update?	PC
LDI imm	Immediate operand	Yes	No	Yes	PC+1
ADD imm	ACC + operand (8-bit)	Yes	No	Yes	PC+1
AND imm	ACC AND operand	Yes	No	Yes	PC+1
STA	ACC is copied to R	No	Yes	No	PC+1
LDR	R value	Yes	No	Yes	PC+1
NOP / unsupported	No architectural data change	No	No	No	PC+1

Z flag rule

Z is set to 1 when the value written into ACC is 0x00; otherwise it is 0. For LDI, ADD, AND and LDR, update Z using the new ACC result. For STA, ACC is not written, so Z remains unchanged. Reset sets Z=1.

Program counter rule

For your Set 4, none of the five implemented instructions is JZ. Therefore, after each executed instruction, PC advances to PC+1. The 4-bit PC addresses 16 ROM entries; the supplied program uses addresses 0 through 9. Locations not specified contain 0x000 (NOP), according to the brief.

Do not add JZ to Set 4: it belongs to other sets. Implement only the five instructions assigned to your ID, as the project brief says.

10. Your required Set 4 ROM program and expected trace

The assignment's Set 4 program is reproduced below. Opcode is the high 4 bits and the operand is the low 8 bits. STA and LDR ignore the operand field; the brief encodes them with operand 0x00.

Address	Instruction	Encoded 12-bit word
0	LDI 0x1E	0x11E
1	STA	0xA00
2	ADD 0x22	0x222
3	AND 0x60	0x460
4	LDR	0xB00
5	ADD 0x01	0x201
6	AND 0x0F	0x40F
7	STA	0xA00
8	LDI 0x00	0x100
9	LDR	0xB00

Expected state after each rising edge

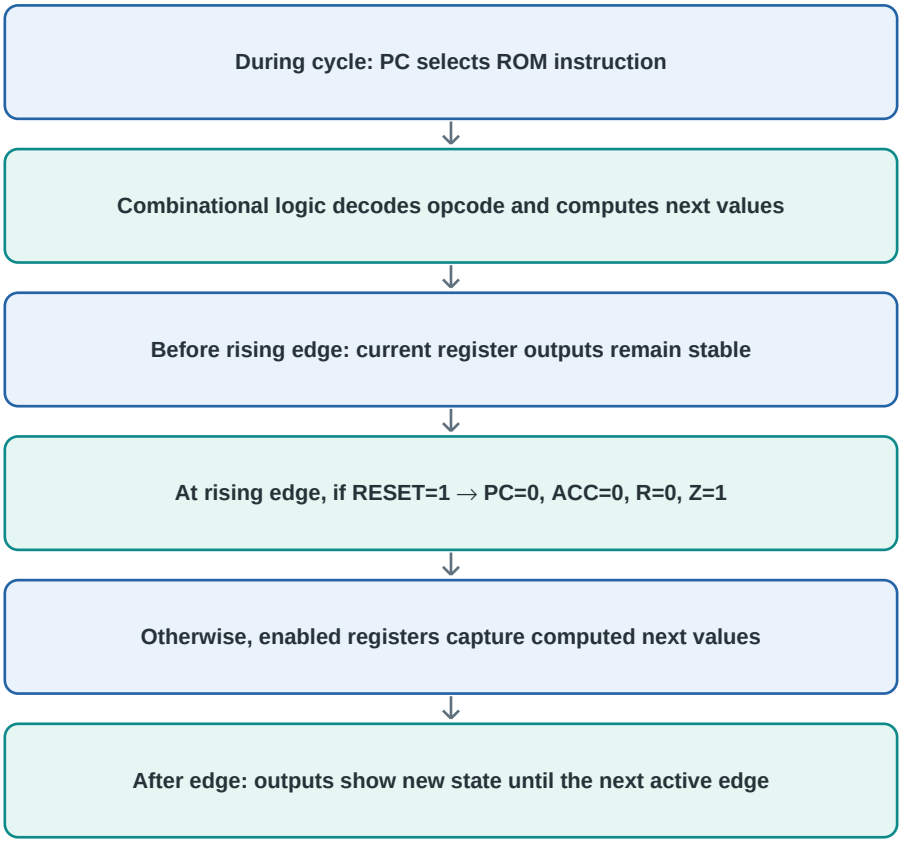
Cycle	PC executed	Instruction	ACC	Z	R	Next PC
0	—	RESET=1	00	1	00	0
1	0	LDI 1E	1E	0	00	1
2	1	STA	1E	0	1E	2
3	2	ADD 22	40	0	1E	3
4	3	AND 60	40	0	1E	4
5	4	LDR	1E	0	1E	5
6	5	ADD 01	1F	0	1E	6
7	6	AND 0F	0F	0	1E	7
8	7	STA	0F	0	0F	8
9	8	LDI 00	00	1	0F	9
10	9	LDR	0F	0	0F	10

Manual trace explanation

1) LDI 1E loads 00011110 into ACC; Z=0. 2) STA saves 1E into R but leaves ACC/Z alone. 3) ADD 22 gives 1E+22=40, so ACC=40. 4) AND 60 gives 40 AND 60 = 40. 5) LDR restores R=1E to ACC. 6) ADD 01 gives 1F. 7) AND 0F gives 0F. 8) STA saves 0F in R. 9) LDI 00 loads zero and sets Z=1. 10) LDR restores 0F and changes Z back to 0.

11. Reset, rising edge and state-update discipline

This is a high-value viva topic. Separate the combinational calculation from the sequential update.



Rules to say in viva

Rule	Correct explanation
Synchronous reset	Reset is acted on only at the rising edge, not immediately when RESET changes.
Reset priority	If RESET=1 at the rising edge, reset wins over instruction execution and all other enables.
Clock edge	Only rising edge commits the new sequential state in this project.
Between edges	Registers hold their values; combinational wires may respond to changing inputs.
Z update	Update only when the instruction says Z is updated; STA preserves Z.
No accidental latch	In combinational VHDL processes, assign every output on every path or give a deliberate default.

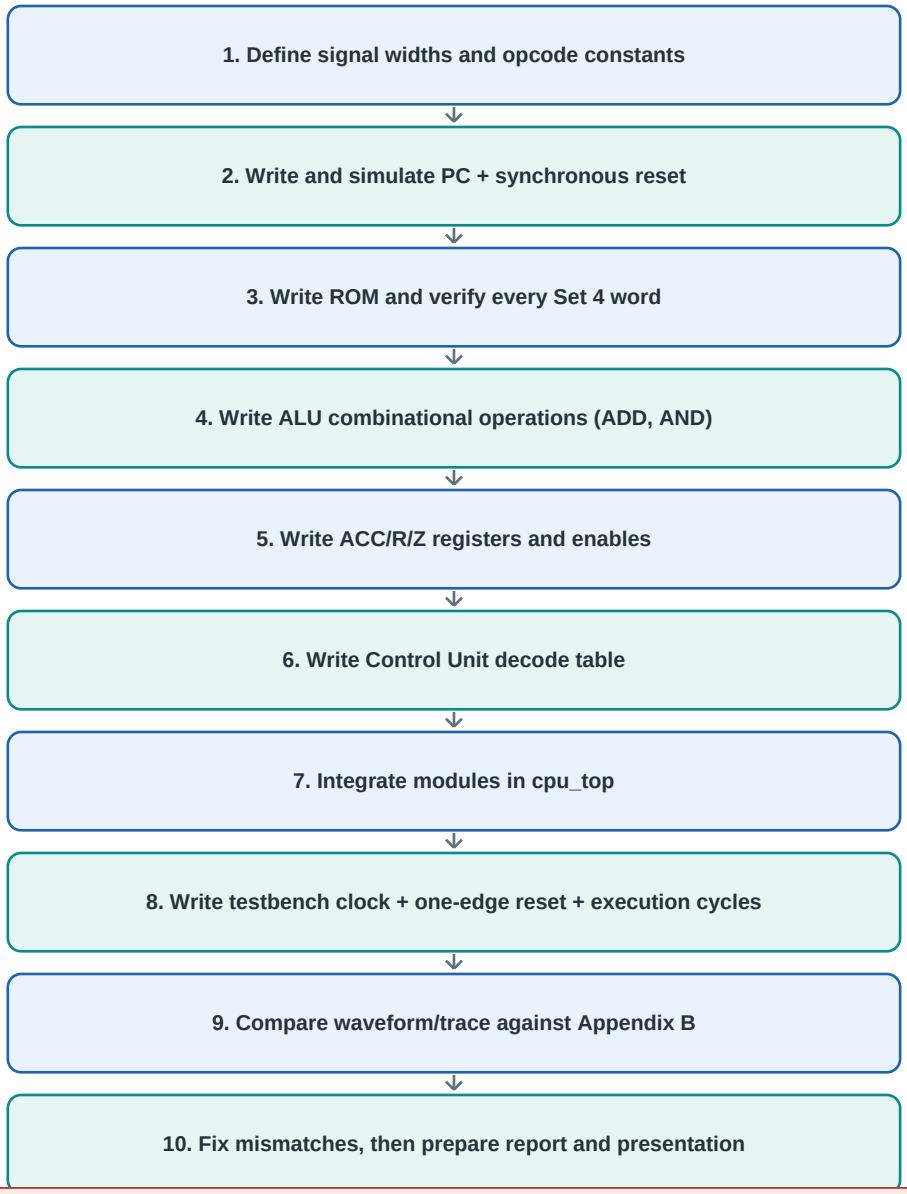
VHDL pattern to remember (conceptual): use `if rising_edge(CLK) then`; inside it, check RESET first; otherwise perform register updates. Use a combinational process for decode/next-state logic and ensure every output is assigned in every branch.

12. Suggested VHDL file/module pipeline

Your assignment explicitly requests separate entities for PC, Program ROM, Control Unit, ALU and ACC/R/Z, connected in a top-level CPU entity. A practical file organization is:

File / entity	Purpose	Main checks
pc.vhd	4-bit PC register and increment/next-PC logic	Synchronous reset to 0; increments by one
program_rom.vhd	16 × 12-bit constant instruction ROM	Set 4 contents exactly match Appendix B
control_unit.vhd	Opcode decode and control signals	Only LDI, ADD, STA, LDR, AND have defined effects
alu.vhd	Combinational ADD and AND operations; support result routing as needed	8-bit arithmetic; correct bitwise result
registers.vhd	ACC, R and Z sequential state	Correct enables, reset and Z rules
cpu_top.vhd	Wire modules together and expose CLK, RESET, Q, Z, PC	No width mismatch or disconnected signals
tb_cpu.vhd	Clock/reset generation and simulation checks	Run enough cycles to finish the 10-instruction program

Implementation sequence

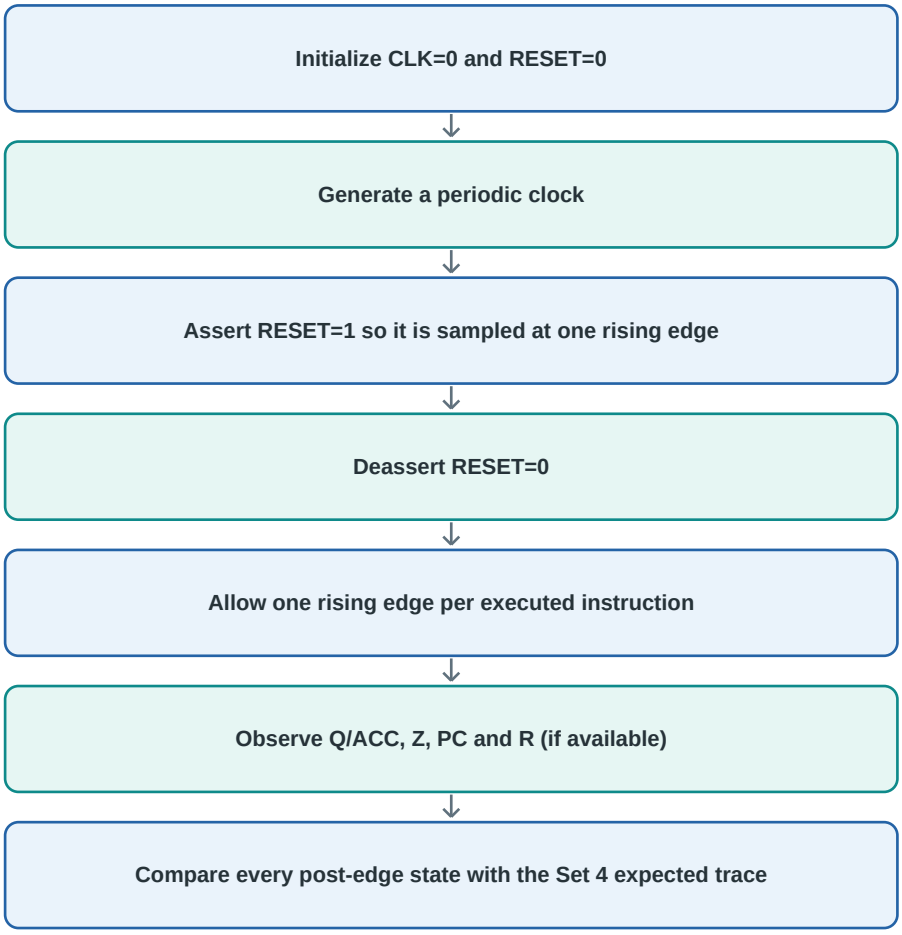


A gate-level NAND-only implementation is not required by the supplied brief unless your instructor separately requests it. For this assignment, the modular VHDL architecture and the exact Set 4 behavior are the acceptance criteria.

13. Testbench and waveform checklist

A testbench is not part of the synthesized CPU hardware. It is a simulation environment that applies clock/reset signals and lets you inspect whether the CPU behaves correctly.

Testbench flow



Waveform signals to include

Signal	Expected observation
CLK	Regular transitions; rising edges are active update points
RESET	High across one rising edge, then low
PC[3:0]	0 after reset; then 1, 2, …, 10 after the listed program
Q[7:0] / ACC	00 → 1E → 1E → 40 → 40 → 1E → 1F → 0F → 0F → 00 → 0F
Z	1 after reset; 0 after LDI 1E; 1 after LDI 00; 0 after LDR 0F
R[7:0]	00 → 1E at first STA → remains 1E until second STA → 0F

Timing caution: The state shown in the assignment tables is the state after the corresponding rising edge. When comparing a waveform, compare post-edge values and account for delta-cycle scheduling in VHDL simulation.

14. Viva questions with concise answers

Use these as spoken answers. Start with the definition, then connect it to your CPU where relevant.

1. What is a NAND gate?

A universal logic gate whose output is NOT(A AND B). It is 0 only when both inputs are 1.

2. Why is NAND called universal?

Because NOT, AND, OR, XOR and more complex Boolean functions can be constructed from NAND gates alone.

3. How do you make NOT using NAND?

Connect both NAND inputs to A. The output is NAND(A,A)=NOT A.

4. How do you make AND using NAND?

First compute NAND(A,B), then NAND that result with itself to invert it.

5. How is OR built from NAND?

Invert A and B with NAND-inverters, then NAND the two inverted signals.

6. What is the difference between combinational and sequential logic?

Combinational output depends on current inputs. Sequential logic stores state, so output also depends on prior state.

7. What is an SR latch?

A bistable storage circuit made with cross-coupled gates; it can set, reset or hold a bit.

8. Why is the NAND SR latch input active-low?

A low input requests set or reset in the usual cross-coupled NAND implementation.

9. What is the forbidden NAND SR latch input combination?

S-bar=0 and R-bar=0; it forces both outputs high and invalidates complementary Q/Q-bar behavior.

10. What is a D latch?

A level-sensitive storage element: when enable is active Q follows D; otherwise it holds the previous value.

11. Latch versus flip-flop?

A latch is level-sensitive; a flip-flop responds to a clock edge.

12. What is a master–slave flip-flop?

Two latches controlled by opposite clock phases; together they provide edge-triggered behavior.

13. What does rising edge mean?

The clock transition from logic 0 to logic 1.

14. What is synchronous reset?

Reset is sampled at the active clock edge; it does not asynchronously clear state between edges.

15. What is the reset state in this project?

At a rising edge with RESET=1: PC=0, ACC=0x00, R=0x00 and Z=1.

16. What is a register?

A group of storage bits. An 8-bit register stores eight bits in parallel.

17. What is a half adder?

A circuit that adds two one-bit inputs, producing Sum=A XOR B and Carry=A AND B.

18. Why does a half adder not replace a full adder in higher bit positions?

A higher bit may receive a carry from the lower bit; a full adder has a carry-in input.

19. How do you build a full adder from half adders?

Use HA1 on A and B, HA2 on HA1's sum and Cin, then OR the two carry outputs.

20. How do you build an 8-bit ripple-carry adder?

Connect eight full adders; each stage's carry-out feeds the next stage's carry-in.

21. Why is the final carry discarded for ADD?

The specification stores only an 8-bit result, so arithmetic wraps modulo 256.

22. What is an accumulator?

A register used to hold the current result and participate in subsequent operations; here it is the CPU output Q[7:0].

23. What does LDI do?

Loads the immediate 8-bit operand into ACC and updates Z.

24. What does ADD do?

Adds the 8-bit immediate operand to ACC, stores the low 8 bits, and updates Z.

25. What does AND do?

Performs bitwise ACC AND operand, stores the 8-bit result and updates Z.

26. What does STA do?

Copies ACC into R; ACC and Z remain unchanged.

27. What does LDR do?

Copies R into ACC and updates Z based on the loaded value.

28. When is Z=1?

When the new ACC value being written is 0x00; reset also sets Z=1.

29. Does STA update Z?

No. The assignment explicitly says Z is unchanged for STA.

30. What is the program counter?

A 4-bit register holding the address of the instruction to fetch from the 16-entry ROM.

31. Why is the PC 4 bits?

Four bits can represent 16 addresses, from 0 through 15.

32. What is the instruction width?

12 bits: 4-bit opcode plus 8-bit operand.

33. What does the control unit do?

Decodes the opcode and generates ALU selection and register/PC control signals.

34. What is the ALU's role in Set 4?

It must support ADD and AND; routing logic also supports loading an immediate and loading R into ACC.

35. Why does LDR not need the ALU to calculate an arithmetic result?

It selects R as the data source for ACC; it is a data-movement operation.

36. What does the ROM contain?

A constant array of 16 12-bit instructions, with the required Set 4 program at addresses 0–9 and NOP words in unused locations.

37. What is a NOP?

No operation on architectural data/state besides the normal PC increment.

38. What is a testbench?

Simulation-only VHDL that drives clock/reset and checks or observes the design.

39. What should the waveform show?

CLK, RESET, PC, ACC/Q, Z and R, with values matching the expected trace after each rising edge.

40. Why modular design?

It separates responsibilities, makes testing easier and helps locate bugs in a specific module.

41. What is a common VHDL latch-inference bug?

Leaving a combinational output unassigned on some branch, causing the tool to infer storage unintentionally.

42. What is the difference between signal assignment and immediate variable update in a VHDL process?

Signal assignments schedule updates; they are not generally visible immediately within the same process activation. This matters when writing sequential and combinational logic.

43. Why must RESET have priority?

The brief explicitly requires reset to override instruction execution at the active rising edge.

44. What is the output after LDI 0x00?

ACC=0x00 and Z=1; R remains unchanged.

45. What is the state after the final LDR in Set 4?

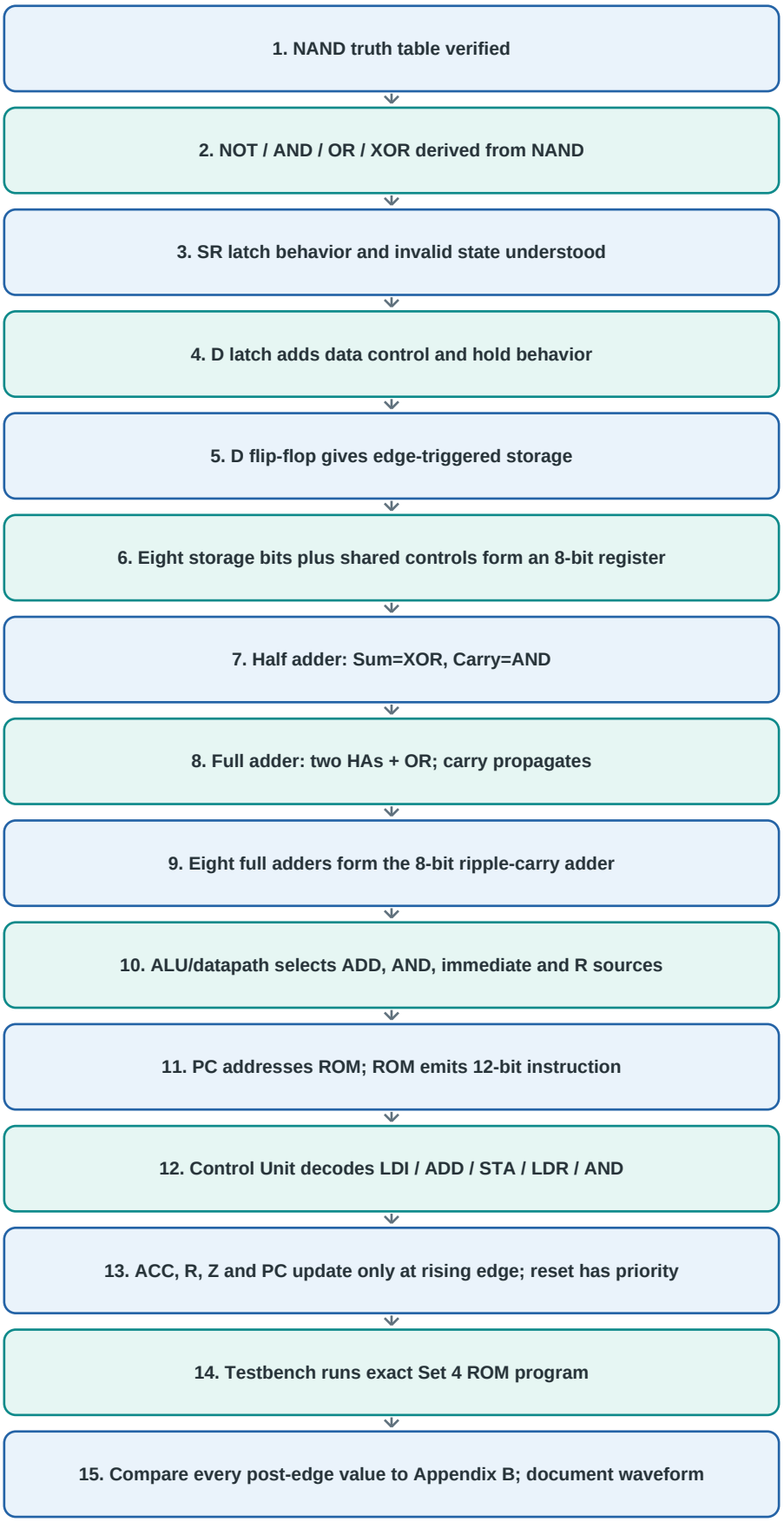
ACC=0x0F, R=0x0F, Z=0 and next PC=10.

46. Is a NAND-only gate implementation explicitly required by the brief?

No. The brief requires separate VHDL modules and correct CPU behavior. NAND-only construction is a conceptual learning path unless separately required by the instructor.

15. Presentation-ready final pipeline (detailed)

You can present this as the end-to-end design story. Explain each stage in order and mention the verification condition at the end of each stage.



Short opening statement for your presentation

“My student ID is 22201054, so my assigned instruction set is Set 4. I implement the mandatory LDI and ADD instructions, plus STA, LDR and AND. The design is a single-cycle 8-bit accumulator CPU in VHDL. The PC selects a 12-bit instruction from ROM, the control unit decodes the opcode, the datapath selects or computes the new accumulator value, and the registers update at the rising clock edge. A synchronous active-high reset initializes PC, ACC, R and Z. I verify the design by running the exact Set 4 ROM program and comparing PC, ACC, R and Z against the expected trace.”

A strong explanation for the modular design

“I separate the design into PC, ROM, control unit, ALU, registers and top-level CPU because each module has one responsibility. I can test the PC increment and reset independently, verify ROM encodings separately, check ALU arithmetic/logic operations with test vectors, then integrate them and use a testbench to compare the full program trace. This makes errors easier to isolate.”

Final pre-viva checklist

Check	Ready?
I can state my ID, Set 4 and five instructions.	■
I can explain NAND-based NOT, AND, OR and XOR.	■
I can distinguish latch, flip-flop and register.	■
I can derive half-adder and full-adder equations.	■
I can explain why 8-bit addition discards carry-out.	■
I can explain synchronous active-high reset and rising edge.	■
I can explain LDI, ADD, STA, LDR, AND and their Z behavior.	■
I know the Set 4 ROM words and expected trace.	■
I can read the waveform for CLK, RESET, PC, ACC, Z and R.	■

Final reminder: The supplied assignment is the authority for the required CPU behavior. If your instructor gives a separate gate-level restriction, follow that additional instruction; otherwise prioritize the exact modular VHDL design, Set 4 ROM program, reset/edge behavior and testbench evidence.